

Lab 3: Keypad Scanning in C

Name: Hemraj Khadka

EE120Lab: Section 07 (Siddharth Mohon)

Date: 09/29/2026

Lab Status

Answer the following questions by putting an "x" in the bracket.

- Yes [x], No []: Complete pre-lab
- Yes [x], No []: Complete basic lab requirements
- Yes [x], No []: Complete something cool

Something Cool Summary: I added an automatic key-repeat feature to my keypad scanning program. The program was designed to repeat a pressed character at approximately 2000 ms intervals when the button was held down. I also added a 16-character buffer to store the previous keypresses and display them on the OLED screen.

Abstract

In this lab, I used the STM32L476 microcontroller to scan a 4×4 keypad and display the pressed keys on an SSD1306 OLED screen. The main purpose of this experiment was to understand keypad scanning, GPIO input and output configuration, software debouncing, and storing keypresses in a buffer. I connected the keypad to GPIOC and used I2C communication to control the OLED display. During my initial test, some keypad buttons were not working correctly. After checking the connections and modifying my code, I successfully tested all 16 buttons, and the OLED displayed their corresponding characters. I also added a 16-character buffer and an automatic key-repeat feature to the program. This experiment helped me understand how the microcontroller detects keypad inputs and displays the results on the OLED screen.

Introduction

A keypad is an input device that allows users to send information to a microcontroller by pressing different buttons. In this lab, I used a 4×4 keypad with 16 buttons and an STM32L476

microcontroller. The keypad has four rows and four columns. When a button is pressed, it connects one row to one column, allowing the microcontroller to identify which button was pressed.

The STM32L476 uses GPIO pins to scan the keypad. I configured PC0, PC1, PC2, and PC3 as outputs for the rows and PC4, PC10, PC11, and PC12 as inputs for the columns. The program checks each row and column to determine the pressed key. A lookup table is then used to convert the detected row and column into the corresponding character.

I also used an SSD1306 OLED display to show the keypad output. The OLED communicates with the STM32L476 through I2C using PB8 for SCL and PB9 for SDA. The program stores the last 16 keypresses in a buffer and updates the OLED when a new key is detected. I added software debouncing to prevent one button press from being counted multiple times due to mechanical contact bouncing. I also included an automatic key-repeat feature when a button is held down.

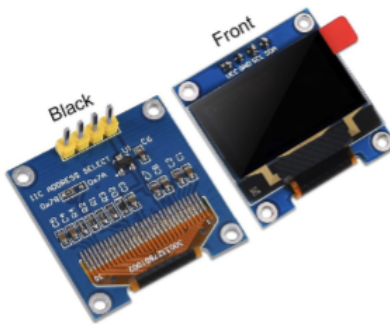


Figure 1: OLED-SSD1306 Display

The main purpose of this experiment was to understand how keypad scanning works and how the microcontroller processes input signals. By testing the keypad and displaying the results on the OLED, I was able to observe how the GPIO configuration, scanning program, and display communication work together.

Theory of operation

A 4×4 keypad contains 16 buttons arranged in four rows and four columns. Each button acts like a switch that connects a row to a column when pressed. Instead of using 16 separate GPIO pins, the STM32L476 only needs eight GPIO pins to detect all 16 buttons. Four pins are used as outputs for the rows, and four pins are used as inputs for the columns.

In this lab, I used PC0, PC1, PC2, and PC3 as row outputs and PC4, PC10, PC11, and PC12 as column inputs. The column inputs are connected to external 2.2 k Ω pull-up resistors, which keep them at logic HIGH when no button is pressed. When a button is pressed and its row is driven LOW, the corresponding column also becomes LOW. The microcontroller reads this change to detect the button.

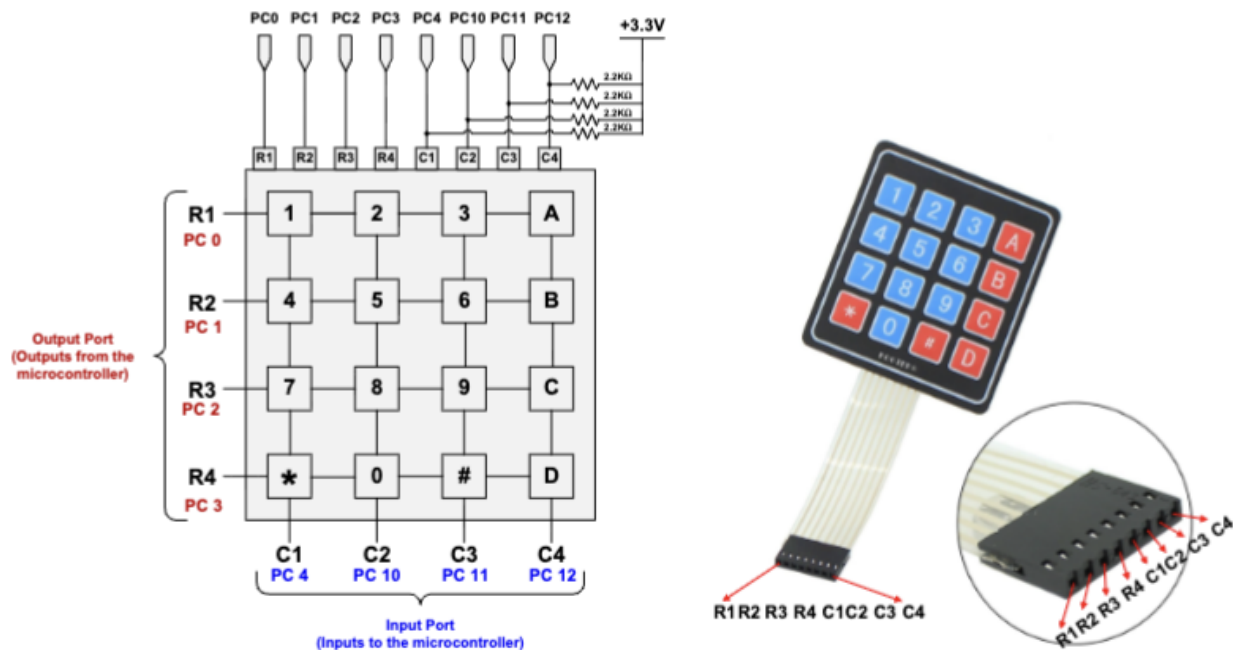


Figure 2: Pinout a 4x4 keypad and pin connection between the Nucleo Kit and Keypad

The keypad scanning program first drives all four rows LOW and checks the column inputs. If all columns remain HIGH, no button is pressed, and the function returns `0xFF`. If a column becomes LOW, the program scans one row at a time to identify the pressed button. It then uses a lookup table to convert the detected row and column into the correct character.

For example, when I press button 5, the program detects row 2 and column 2 of the physical keypad, using one-based numbering. The lookup table then returns the character '5', which is displayed on the OLED screen.

The GPIO pins were configured using the STM32L476 registers. First, I enabled the GPIOC clock using the RCC AHB2ENR register. I then configured PC0–PC3 as outputs and PC4, PC10, PC11, and PC12 as inputs using the GPIOC MODER register. The program uses the ODR register to control the keypad rows and the IDR register to read the column signals. These registers allow the microcontroller to scan the keypad and identify which button is pressed.

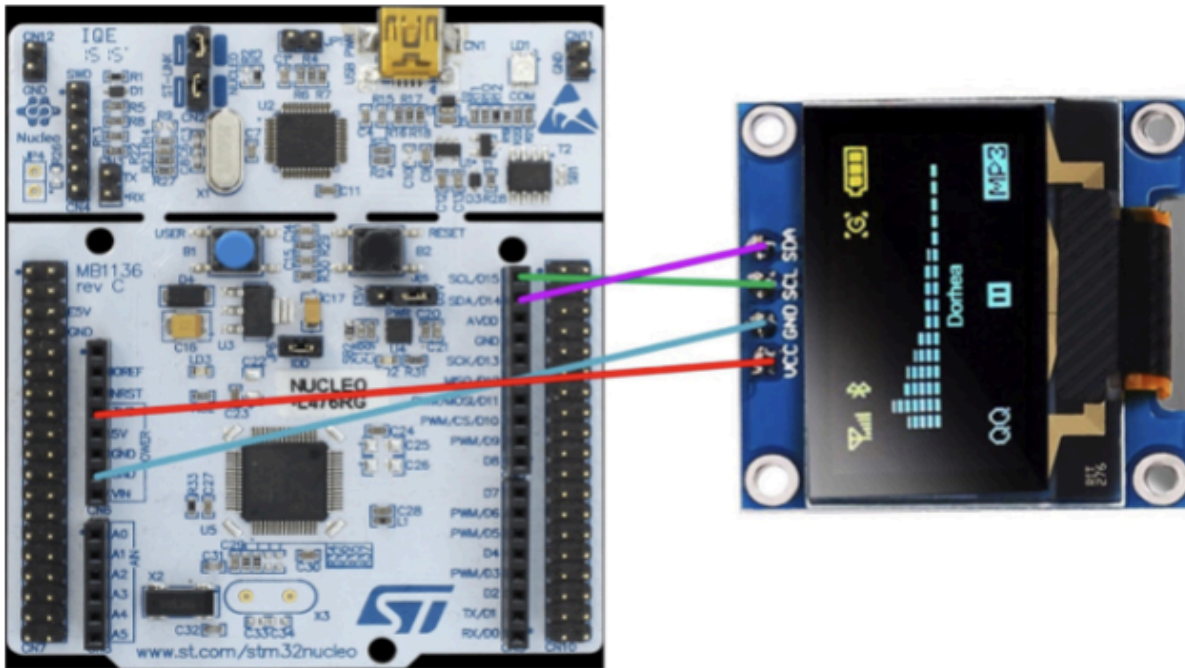


Figure 3:STM32L476 NUCLEO Board and SSD1306 OLED Connection Diagram.

The SSD1306 OLED communicates with the STM32L476 through I2C. I used PB8 as the clock line (SCL) and PB9 as the data line (SDA). The program stores the last 16 keypresses in a buffer and updates the OLED whenever a new key is registered. When the buffer becomes full, the oldest character is removed to make space for the newest character.

I also used a 20 ms software debounce delay to check whether the button reading was stable. This helps prevent unwanted characters caused by mechanical contact bouncing. For the additional feature, I added automatic key repeat with an approximate 2000 ms interval while a button remains pressed.

The keypad scanning process continues inside an infinite loop, allowing the microcontroller to detect new keypresses and update the OLED display continuously.

Procedure

First, I connected the 4×4 keypad to the STM32L476 NUCLEO board. I connected the four keypad rows to PC0, PC1, PC2, and PC3 and the four columns to PC4, PC10, PC11, and PC12. The column inputs used external 2.2 kΩ pull-up resistors to keep the signals HIGH when no

button was pressed. I also connected the SSD1306 OLED display to the NUCLEO board using 3.3 V, GND, PB8 for SCL, and PB9 for SDA. The NUCLEO board was powered through the USB connection.

Next, I opened the project in Keil μ Vision and configured the GPIOC registers in `keypad.c`. I enabled the GPIOC clock and configured PC0–PC3 as row outputs and PC4, PC10, PC11, and PC12 as column inputs. I used a lookup table to identify each button based on its row and column position. The `keypad_scan()` function checked the column inputs and scanned each row to determine which button was pressed. When no button was pressed, the function returned `0xFF`.

After configuring the keypad, I initialized the OLED display using I2C communication. In `main.c`, I called `Keypad_Pin_Init()`, `OLED_I2C_Init()`, and `ssd1306_Init()` before starting the keypad scanning loop. I programmed the OLED to display "Keypad:" on the first line and the detected characters below it.

For the next part, I added a 16-character buffer to store the previous keypresses. When a new button was pressed, the program added its character to the buffer and updated the OLED. When the buffer became full, the oldest character was removed to make space for the newest character. I also added a 20 ms software debounce delay to reduce unwanted repeated readings caused by mechanical contact bouncing.

For the additional feature, I added automatic key repeat. The program was designed to register a held button again at approximately 2000 ms intervals. I also used the onboard LED to indicate when a key was registered.

During my initial test, some keypad buttons were not responding correctly, even though the OLED was displaying characters. I checked the keypad wiring and reviewed the GPIOC configuration in `keypad.c`. I also checked the row and column scanning logic and updated the lookup table to match the physical keypad labels. After making these changes, I compiled the program again and uploaded it to the STM32L476 board. During the final test, all 16 buttons worked correctly and displayed their corresponding characters on the OLED.

Results and Discussion

During my initial test, I faced some difficulties with the keypad and OLED display. The OLED was working and displaying the keypad characters, but some buttons were not responding correctly. At first, I noticed that only a few buttons were working, while others did not display any characters. I checked the wiring connections between the keypad and STM32L476 board to make sure the rows and columns were connected to the correct GPIO pins.

Another difficulty I faced was with the keypad scanning code. Some buttons were not being detected correctly because the row and column scanning configuration needed to be checked. I reviewed the `keypad.c` file, GPIO settings, and lookup table. I made changes to the code and tested the keypad again. During troubleshooting, I also had difficulty getting the ST-LINK connection to work in Keil μ Vision, which delayed uploading and testing the program.

After making the necessary changes, I successfully compiled and uploaded the updated program. I tested all 16 buttons, including 0–9, A–D, *, and #. This time, all buttons responded correctly, and the OLED displayed their corresponding characters. The experimental video shows the keypad responding to button presses and displaying the results on the OLED screen.

I also added a 16-character buffer to store the previous keypresses. When I pressed a new button, its character was added to the OLED display. The program was designed to remove the oldest character when the buffer became full. I used a 20 ms software debounce delay to reduce unwanted repeated readings. For the additional feature, I included automatic key repeat with an approximate 2000 ms interval when a button was held down.

Test	Result
OLED initialization	Displayed “Keypad:”
Initial keypad test	Some buttons did not respond
GPIO and code troubleshooting	Reviewed wiring, scanning logic, and lookup table
Final keypad test	All 16 buttons responded correctly
OLED character display	Displayed the corresponding pressed characters
Keypress buffer	16-character buffer implemented
Software debounce	20 ms delay implemented
Automatic key repeat	Approximately 2000 ms interval programmed

Table 1: Experimental Keypad Scanning Results



Figure 4: Experimental 4×4 Keypad and OLED Display Output.

The final results agreed with the theory of operation. When a button was pressed, it connected one row to one column, allowing the microcontroller to identify the pressed key. The lookup table converted the detected row and column into the correct character, which was then displayed on the OLED.

The main difficulty in this experiment was getting all the keypad buttons to respond correctly. I learned that even when the program compiles without errors, the hardware connections and GPIO configuration still need to be checked carefully. After troubleshooting the connections and modifying the code, I was able to get the expected output. This experiment helped me understand how keypad scanning, GPIO configuration, and OLED communication work together.

Post-Lab: Question and Answer

1. Evaluating Key Press Detection and Ghosting

My keypad scanning program was designed to detect one button at a time by checking the rows and columns. During my final test, all 16 buttons worked correctly when pressed individually. However, my program was not designed to identify multiple buttons pressed at the same time. The scanning function returns the first detected key, so it cannot identify all the individual keys during simultaneous keypresses.

Ghosting can happen when multiple buttons are pressed together, causing the microcontroller to detect a button that was not actually pressed. My program uses software debouncing to reduce unwanted repeated inputs, but debouncing does not prevent ghosting. I did not implement a separate ghosting prevention method. To improve the program, I would add multi-key detection and use a keypad with isolation diodes to prevent false key detection.

2. Reflection

The most challenging part of this lab was getting all 16 keypad buttons to work correctly. During my initial test, some buttons were not responding, even though the OLED was displaying characters. I checked the keypad connections, GPIO configuration, and scanning code to find the problem. I also had difficulty with the ST-LINK connection while uploading the program.

I reviewed the row and column scanning logic and made changes to my code. After compiling and uploading the updated program, I tested the keypad again. This time, all 16 buttons displayed their correct characters on the OLED screen.

If I had more time, I would improve the program to detect multiple buttons pressed at the same time and prevent ghosting. I would also test the automatic key-repeat timing more carefully. This lab helped me understand the importance of checking both hardware connections and software configuration when troubleshooting a microcontroller project.

Learning Reflection

The most useful part of this lab was learning how the STM32L476 detects a button by scanning the keypad rows and columns. At first, I had difficulty getting all the buttons to work, so I checked the wiring and made changes to my code. After testing again, all 16 buttons displayed the correct characters on the OLED. I also learned how the 16-character buffer stores previous keypresses and why software debouncing is needed. If I repeat this lab, I would test each row and column separately before running the complete program. This would help me find wiring or code problems more quickly.

Conclusion

In this lab, I successfully used the STM32L476 microcontroller to scan a 4×4 keypad and display the pressed characters on an SSD1306 OLED screen. During my initial test, some keypad buttons were not responding correctly. After checking the GPIO connections and modifying the keypad scanning code, I was able to get all 16 buttons working correctly. I also implemented a 16-character buffer, 20 ms software debouncing, and an automatic key-repeat feature with an approximate 2000 ms interval. The final results confirmed that the microcontroller could detect the pressed buttons using row and column scanning and display the corresponding characters on the OLED. This lab helped me understand GPIO configuration, keypad scanning, I2C communication, and the importance of troubleshooting both hardware and software to get the expected output.

References

San José State University. “EE 120: Microprocessor Based System Design.” Canvas, 2026. [SJSU EE 120 Canvas Lab 3 Course Files](#). Accessed 21 Sept. 2026

Y. Zhu, *Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C*, 4th ed., Chapter 15.9, “Keypad Scanning.

STMicroelectronics. *STM32L476xx [Reference Manual \(RM0351\)](#): Advanced Arm-Based 32-Bit MCUs*.

STMicroelectronics. [STM32 Nucleo-64 Boards User Manual \(UM1724\)](#).

[Appendix A - Source Code](#)

[Appendix B – Experimental Test Video and Final Keypad Output on the OLED Display](#)